

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 4: **Desain Basis Data, Migrasi, dan Seeding**

Skema, Riwayat Perubahan Skema, dan Kecepatan Query

Yang Akan Kamu Pelajari

1. Menjelaskan konsep **skema** dan relasi antar tabel lewat **foreign key**
2. Memahami **migration** sebagai riwayat perubahan skema yang bisa dijalankan ulang, dan **seeder** untuk mengisi data awal dalam jumlah besar
3. Menjelaskan konsep **index** dan cara memverifikasi dampaknya terhadap kecepatan query

Slide ini membahas konsep. Merancang skema, menulis migration, dan seeder untuk Simple POS dikerjakan di jobsheet praktikum.

Bagian 1

Gudang Berkatalog vs Tanpa Katalog

Gudang dengan katalog kartu

- Setiap barang ada di rak berlabel
- Katalog mencatat rak mana menyimpan barang apa
- Cari barang: baca katalog, langsung ke rak yang tepat

Gudang tanpa katalog

- Barang tetap ada di suatu tempat
- Mencari berarti menyusuri rak satu per satu
- Pada gudang besar, ini menyita banyak waktu

Skema tabel basis data adalah tata letak rak itu sendiri; index adalah katalog kartunya. Tanpa index yang tepat, mesin basis data tetap bisa menemukan datanya, hanya saja dengan menyusuri seluruh tabel satu per satu.

Skema: Rancangan Struktur Tabel

Skema: rancangan struktur tabel, yaitu nama kolom, tipe data, dan batasan (nullable, default, unique) yang berlaku pada setiap barisnya.

- Skema digambar di atas kertas sebelum satu baris migration pun ditulis
- Setiap kolom punya tipe data (angka, teks, tanggal) dan batasan yang menjaga data tetap valid

Foreign Key: Menjaga Relasi Tetap Konsisten

Foreign Key: kolom pada satu tabel yang menunjuk ke `id` baris pada tabel lain, menjaga agar sebuah baris tidak pernah menunjuk ke baris induk yang tidak ada.

authors (satu) → **articles (banyak)**

- Satu `authors` punya banyak `articles` ; setiap baris `articles` menyimpan `author_id` yang menunjuk balik ke penulisnya

Contoh Skema: Relasi Satu ke Banyak

Tabel	Kolom Utama
authors	id , name
articles	id , author_id (fk), title , published_at
comments	id , article_id (fk), body

- Satu authors punya banyak articles
- Satu articles punya banyak comments
- Relasinya cukup lurus untuk digambar di atas kertas sebelum menulis kode

Nilai yang Disimpan, Bukan Dihitung Ulang

- Sebagian kolom sengaja menyimpan nilai pada satu titik waktu, bukan menghitungnya ulang setiap kali dibaca
- Contoh: jumlah komentar saat artikel dipublikasikan, atau harga yang berlaku saat transaksi terjadi
- Nilai itu harus tetap sama persis seperti kondisi saat kejadian itu terjadi, meski data sumbernya berubah kemudian

Pertemuan validasi & keamanan nanti membahas mengapa nilai seperti ini tetap wajib dihitung ulang di server saat disimpan, bukan sekadar dipercaya dari input.

Menulis Migration untuk Sebuah Tabel

```
Schema::create('articles', function (Blueprint $table) {  
    $table->id();  
    $table->foreignId('author_id')->constrained();  
    $table->string('title');  
    $table->timestamps();  
});
```

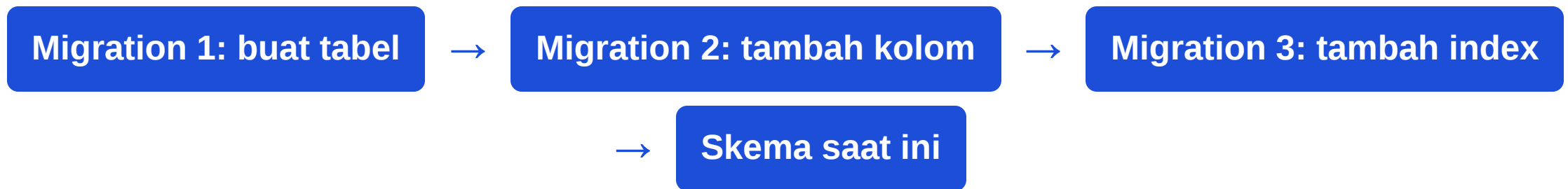
- `foreignId('author_id')` membuat kolom `author_id` bertipe unsigned big integer
- `->constrained()` menambahkan foreign key yang menunjuk ke `authors.id` berdasarkan konvensi penamaan Laravel
- Constraint ini menjaga integritas data, tapi belum tentu berarti kolomnya sudah punya index (lihat Bagian 3)

Bagian 2

Riwayat perubahan skema, dan mengisi data awal

Migration: Riwayat Skema yang Bisa Dijalankan Ulang

Migration: berkas PHP yang mendeskripsikan perubahan struktur tabel (membuat, mengubah, menghapus kolom) secara terprogram, sehingga skema basis data punya riwayat yang bisa dijalankan ulang di komputer lain.



Mirip riwayat commit git untuk kode: migration adalah riwayat commit untuk skema basis data, dijalankan berurutan sesuai stempel waktu nama berkasnya.

Aturan Emas Migration: Jangan Ubah, Tambah Baru

Kalau skema perlu berubah, buat migration baru, jangan mengedit migration lama yang sudah pernah dijalankan. Migration lama yang diedit setelah dijalankan membuat riwayat skema di komputer lain jadi tidak sinkron dengan riwayat di komputermu.

- Perlakukan migration seperti version control untuk skema: perubahan baru selalu berarti berkas migration baru

Seeder: Mengisi Data Awal

Seeder: kelas PHP yang mengisi tabel dengan data awal atau data uji, dijalankan lewat `db:seed` atau sebagai bagian dari `migrate:fresh --seed`.

- Berguna untuk data contoh saat pengembangan, dan data uji berskala nyata untuk latihan performa

Seeding Skala Besar: Satu-per-Satu vs Bulk Insert

`Model::create()` di dalam loop

- Satu query `INSERT` per baris
- Overhead: membentuk objek Eloquent, memicu event model
- Lambat untuk ratusan atau ribuan baris

`DB::table()->insert()` dalam batch

- Satu pernyataan `INSERT` untuk banyak baris sekaligus
- Jauh lebih sedikit round-trip ke basis data
- `array_chunk()` membagi batch karena sebagian mesin basis data membatasi jumlah baris per `INSERT`

```
$articles = [];  
foreach ($authorIds as $authorId) {  
    for ($i = 0; $i < 30; $i++) {  
        $articles[] = [  
            'author_id' => $authorId,  
            'title' => fake()->sentence(),  
            'created_at' => now(),  
            'updated_at' => now(),  
        ];  
    }  
}
```

Konsistensi Antar Tabel: `DB::transaction()`

Saat satu operasi menyentuh lebih dari satu tabel yang harus konsisten satu sama lain (mis. menyimpan sebuah pesanan beserta baris-baris itemnya), bungkus dalam satu `DB::transaction(function () { ... })`. Kalau prosesnya gagal di tengah jalan, seluruh perubahan di dalam blok itu dibatalkan bersama, sehingga data tidak pernah berakhir setengah tersimpan.

Bagian 3

Membuktikan dampaknya dengan EXPLAIN QUERY PLAN

Index: Struktur Data untuk Pencarian Cepat

Index: struktur data tambahan yang dibangun mesin basis data di atas satu atau beberapa kolom, mempercepat pencarian baris pada kolom itu tanpa harus memindai seluruh tabel.

Tanpa index (SCAN)

- Mesin basis data memeriksa tiap baris satu per satu
- Waktu bertambah linear seiring tabel bertambah

Dengan index (SEARCH)

- Mesin basis data melompat langsung ke baris yang relevan
- Hampir tidak terpengaruh ukuran tabel

constrained() Tidak Selalu Berarti Ada Index

- Di beberapa mesin basis data (mis. MySQL), `foreignId()->constrained()` otomatis membuat index sebagai efek samping
- Di SQLite, `constrained()` hanya menambahkan foreign key constraint, bukan index
- Constraint menjaga integritas data (menolak insert yang menunjuk ke baris tak ada), tapi tidak mempercepat pencarian

Membuktikan dengan **EXPLAIN QUERY PLAN**

- **EXPLAIN QUERY PLAN** : perintah SQLite yang menampilkan strategi mesin basis data untuk menjalankan sebuah query, tanpa benar-benar menjalankannya

```
sqlite3 database/database.sqlite \  
"EXPLAIN QUERY PLAN SELECT * FROM articles WHERE author_id = 3;"
```

- **Sebelum index**: hasilnya memuat **SCAN articles**
- **Sesudah index ditambahkan**: hasilnya berubah menjadi **SEARCH articles**
USING INDEX articles_author_id_index

Kalau perintah `sqlite3` tidak dikenali, jalur alternatifnya lewat `php artisan tinker` memakai `DB::select("EXPLAIN QUERY PLAN ...")` .

Index Bukan Optimasi Prematur

Pada tabel berisi puluhan baris, perbedaan SCAN dan SEARCH nyaris tidak terasa. Pada tabel berisi ribuan baris, SCAN berarti setiap baris ikut diperiksa pada setiap query, dan waktu itu bertambah linear seiring tabel bertumbuh. Index bukan optimasi prematur; ia perbaikan atas bug performa yang sudah nyata begitu data mendekati skala produksi.

Menerapkan pada Simple POS

- Konsep skema, migration, dan index ini akan kamu terapkan langsung pada studi kasus Simple POS di jobsheet praktikum
- Merancang tabel kategori-produk-transaksi, menulis seeder berskala ratusan-ribuan baris, dan membuktikan index lewat `EXPLAIN QUERY PLAN`

Kode lengkap: github.com/se-polinema/simple-pos , branch `chapter-04`

Rangkuman

- Skema tabel didefinisikan lewat migration; migration adalah riwayat perubahan yang bisa dijalankan ulang di komputer lain, dan aturan emasnya: tambah migration baru, jangan edit yang lama
- Foreign key menjaga integritas relasi satu ke banyak antar tabel; seeder mengisi data awal, dan seeding skala besar memakai `DB::table()->insert()` dalam batch, jauh lebih efisien daripada `Model::create()` satu per satu
- Index mempercepat pencarian pada kolom yang sering difilter; foreign key constraint tidak otomatis berarti ada index, tergantung mesin basis data yang dipakai
- `EXPLAIN QUERY PLAN` membuktikan dampak index secara langsung: dari `SCAN` (memindai semua baris) menjadi `SEARCH` (melompat ke baris yang relevan)

Referensi & Diskusi

Dokumentasi resmi Laravel (Migrations, Query Builder, Seeding)

Kode lengkap: `github.com/se-polinema/simple-pos`

Pertemuan berikutnya: ORM & Relasi Data